

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

**Факультет компьютерных наук
Департамент программной инженерии**

Пугач Виктория Павловна, группа МСПИН251

R2+R3 Проектирование и тестирование программной системы
для курсовой работы

Архитектура образовательной веб-платформы для совместного программирования с
поддержкой CRDT и генерацией учебных подсказок
**(Design of a Web-Based Educational Platform for Collaborative Programming with
CRDT and Hint Generation)**

Руководитель:
доцент департамента программной инженерии, к.т.н,
Брейман Александр Давидович

07/03/2026

Москва, 2026 г.

Оглавление

1 Проектирование системы	3
1.1 Общее описание системы.....	3
1.2 Декомпозиция системы	3
1.2.1 Взаимодействие между модулями.....	4
1.3 Описание бизнес-процессов	5
1.3.1 Ключевые бизнес-процессы (BP)	5
1.4 Архитектура системы	7
1.4.1 Выбранная архитектура.....	7
1.4.2 Архитектурные диаграммы (C4) + ER-диаграмма БД.....	7
1.5 Архитектура инфраструктуры.....	8
1.5.1 Диаграмма развертывания (Deployment Diagram).....	8
1.6 Технологический стек	11
2 Тестирование системы	12
2.1 Введение	12
2.1.1 Область и границы тестирования	12
2.1.2 Связь тестирования с целями проекта, указанными в ТЗ.....	12
2.2 Объекты тестирования (Test Items)	12
2.3 Тестируемые функции.....	13
2.3.1 Основные функции системы, которые будут тестироваться	13
2.3.1 Исключенные из тестирования на этапе MVP функции.....	13
2.4 Стратегия тестирования	13
2.4.1 Виды тестирования	13
2.4.2 Ручное и автоматизированное тестирование	14
2.4.3 Уровни и методы тестирования	14
2.5 Критерии успешности / неуспешности (Acceptance Criteria).....	14
2.5.1 Критерии успешного прохождения	14
2.5.2 Критерии отказа и приостановки тестирования	15
2.6 Результаты тестирования (Test Deliverables).....	15
2.6.1 Ключевые тест-кейсы	15
2.6.2 Шаблон отчета об ошибках (defect reports)	18
2.6.3 Итоговый отчёт о качестве системы.....	18
3 Список источников	20

1 Проектирование системы

1.1 Общее описание системы

Проектируемая система – это образовательная веб-платформа для совместного программирования детей (6–17 лет) с бесконфликтным редактированием на основе CRDT и адаптивными ИИ-подсказками. Платформа разрабатывается по заказу образовательной организации «Инжиниум» и ориентирована на замену недоступных зарубежных EdTech-инструментов.

Таблица 1 – Связь проектных решений с требованиями, указанными в ТЗ

Требование ТЗ	Архитектурное решение
ФТ 1–5: редактор кода, сессии, CRDT, история	Сервис совместного редактирования (Realtime Sync Service)
ФТ 6–11: AST-анализ, обнаружение ошибок, подсказки	Сервис анализа кода и подсказок (Hint Service)
ФТ 12–15: аутентификация, проекты, геймификация, API	Сервис управления обучением (API Gateway + LMS Module)
НФТ 1: задержка < 100 мс	WebSocket (WSS) + CRDT (Yjs) + Redis Pub/Sub
НФТ 2: горизонтальное масштабирование	Контейнеризация (Docker/Kubernetes), stateless-сервисы
НФТ 3: офлайн-устойчивость	Буферизация CRDT-операций на клиенте (Yjs)
НФТ 4: детский интерфейс	React SPA, геймификация
НФТ 5: безопасность HTTPS/WSS	TLS Termination на уровне Ingress
НФТ 6–7: модульность, DDD	Три ограниченных контекста

1.2 Декомпозиция системы

Декомпозиция системы выполнена на основе принципов предметно-ориентированного проектирования (DDD), что напрямую закреплено в НФТ-7 ТЗ. Ключевым основанием для выбора DDD послужила разнородность двух предметных областей системы: домен совместного редактирования оперирует понятиями распределённых структур данных, тогда как домен педагогического анализа работает с понятиями AST, типа ошибки, учебной подсказки и профиля ученика. «Языки» принципиально различны, и их смешение в единой модели привело бы к росту связности и деградации сопровождаемости. Выделение трёх ограниченных контекстов (Bounded Context, BC) позволяет развивать модули независимо.

1.2.1 Основные модули (контексты) системы, их назначение и зоны ответственности

Таблица 2 – BC-1: Совместное редактирование (Collaborative Editing)

Модуль	Назначение	Ответственность	ФТ/НФТ
Веб-редактор кода	Браузерный текстовый редактор с подсветкой синтаксиса (в MVP для Python).	Показ актуального документа; подсветка синтаксиса; отображение подсказок на строках кода.	ФТ-1, ФТ-9
WebSocket Handler	Серверный слой постоянных WSS-соединений. Единовременная передача сообщений участникам.	Поддержка WSS, маршрутизация к CRDT/Awareness, broadcast.	ФТ-3, НФТ-1
CRDT-Engine	Реализация CRDT (Yjs/YATA), хранение Y.Doc, применение/генерация Update-операций, гарантированная конечная согласованность. Yjs выбран как проверенная CRDT-библиотека для JS/Node.js.	Конвергенция при любом порядке доставки; локальная буферизация при разрыве соединения.	ФТ-3, НФТ-3
Session Manager	Жизненный цикл сессии: создание, invite-ссылка, участники, завершение; хранение метаданных в БД.	Создание/закрытие сессий; генерация invite-кодов; список участников с метками активности.	ФТ-2, ФТ-4
Awareness Protocol	Обмен метаданными: курсор и выделение участников, отдельно от CRDT-потока.	Синхронизация cursor/selection; отображение аватаров участников.	ФТ-4
Snapshot Service	Периодическое сохранение снапшотов, поддержка отката. Триггер по числу операций	Создание снапшотов; API отката к выбранной версии.	ФТ-5
Message Broadcaster	Pub/Sub-слой для рассылки CRDT-операций между узлами при горизонтальном масштабировании.	Доставка операций всем нодам кластера и их клиентам.	НФТ-2

Таблица 3 – ВС-2: Педагогический анализ (Pedagogical Analysis)

Модуль	Ответственность	Ответственность	ФТ/НФТ
AST Analyzer	Строит структурированное представление кода для дальнейшей проверки. Работает в фоне с небольшой задержкой, чтобы не анализировать каждое нажатие клавиши. Парсер Tree-sitter.	Поддерживать актуальное дерево разбора; корректно обрабатывать код с ошибками; поддерживать Python.	ФТ-6
Error Detector	По дереву кода находит типовые ошибки новичков: незакрытые скобки или кавычки, необъявленные переменные, другие синтаксические ошибки.	Определять тип ошибки и место; передавать результат генератору подсказок и в редактор.	ФТ-7
Hint Generator	Генерирует понятное ребёнку объяснение по типу ошибки по шаблонам, при необходимости с LLM.	Создавать текст подсказки и её привязку к строке.	ФТ-8
Hint Personalizer	Учитывает историю ошибок ученика, чтобы адаптировать подсказки.	Хранить историю запросов подсказок; передавать профиль ошибок генератору; менять детализацию подсказки.	ФТ-8

Таблица 4 – ВС-3: Управление обучением (Learning Management)

Модуль	Ответственность	Ответственность	ФТ/НФТ
Auth Service	Регистрация, проверка учётных данных, выдача JWT; хэширование паролей.	CRUD учёток; валидация токенов; контроль доступа к сессиям.	ФТ-12, НФТ-5
User Profile & Gamification	Хранение прогресса; начисление достижений.	Триггеры по событиям; профиль пользователя для UI.	ФТ-14
Project Registry	CRUD сессий пользователя; раздел «Мои проекты/сессии».	Список сессий; перезаход в сессию по ссылке.	ФТ-13
Analytics Collector	Сбор обезличенных метрик: типы ошибок, длительность сессий, число hint-запросов.	События без РП; агрегированные отчёты для администратора.	Таб. 1 ТЗ
REST API Gateway	Единая точка входа, проверка JWT, маршрутизация к сервисам.	Маршрутизация, аутентификация; публичный REST API для интеграций.	ФТ-15

1.2.1 Взаимодействие между модулями

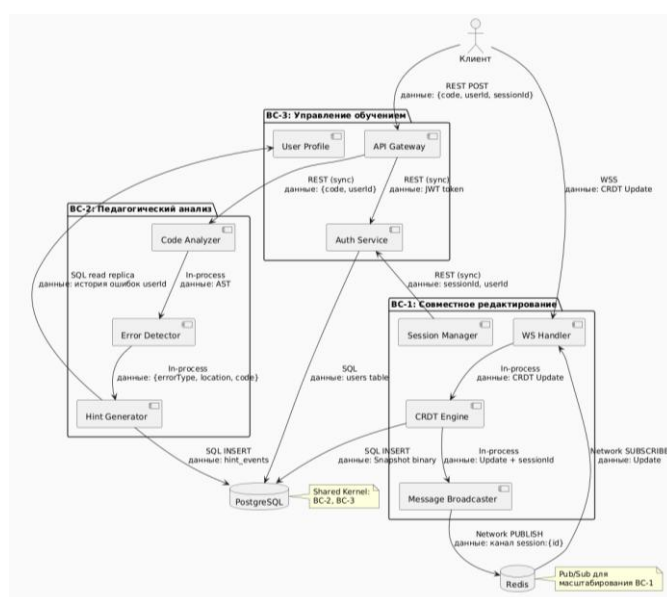


Рисунок 1 – Взаимодействие между модулями системы

Принципы интеграции (DDD):

- Границы контекстов: ВС-1, ВС-2, ВС-3 общаются только через API Gateway или события; прямых импортов моделей нет.
- Shared Kernel: только общая БД для ВС-2 и ВС-3; ВС-1 пишет снапшоты, но не читает.
- Anti-Corruption Layer: ВС-3 Gateway преобразует внешние REST-запросы в доменные вызовы к ВС-1/ВС-2.
- Pub/Sub внутри ВС-1: Redis для горизонтального масштабирования; изоляция CRDT-синхронизации от остальных контекстов.

1.3 Описание бизнес-процессов

1.3.1 Ключевые бизнес-процессы (BP)

BP-1: Совместное редактирование кода в сессии (ФТ 1–5). Ученики одновременно работают над общим кодом в реальном времени

BP-2: Получение подсказки при ошибке (ФТ 6–11). Система автоматически помогает ученику исправить ошибку

BP-3: Преподаватель управляет сессией. Преподаватель создаёт сессию и приглашает учеников

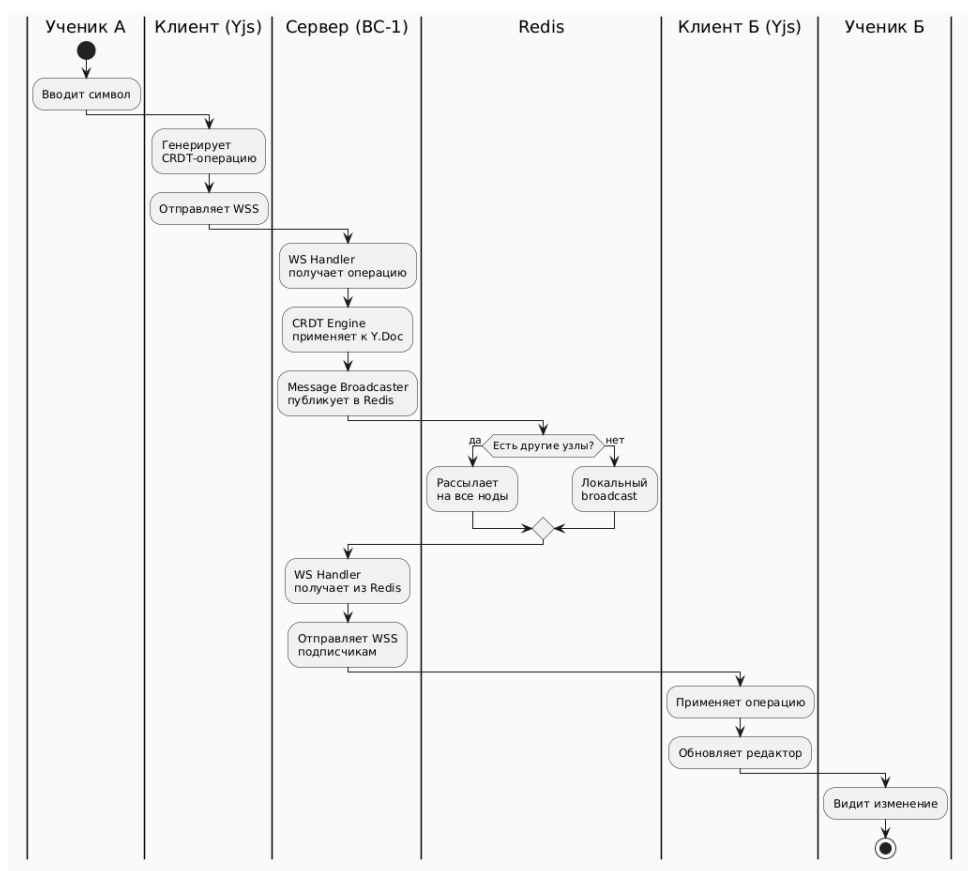


Рисунок 2 – Поток выполнения для BP-1

BP-1

Связь с компонентами: Процесс реализован компонентами ВС-1. Задействованы модули Session Manager для управления участниками и Snapshot Service для сохранения истории изменений.

Поток данных: Ученик вводит символ → Yjs генерирует CRDT Update → WS Handler → CRDT Engine применяет к Y.Doc → Message Broadcaster публикует в Redis канал session:{id} → все ноды получают Update → рассылка участникам через WSS → синхронизированное отображение у всех учеников. Задержка передачи данных < 100 мс (НФТ-1).

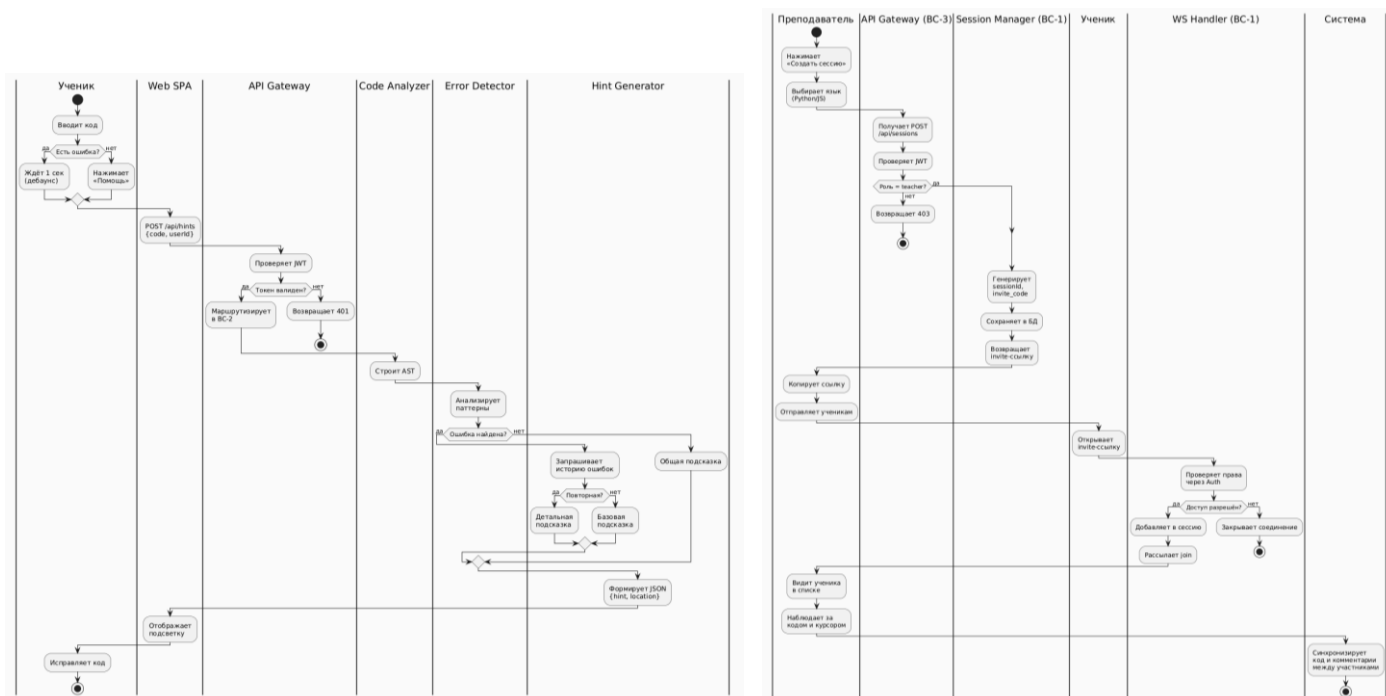


Рисунок 3 – Потоки выполнения для BP-2 и BP-2

BP-2

Связь компонентов: Процесс охватывает BC-2. Интеграция через REST API Gateway, который маршрутизирует запросы между контекстами.

Поток: Ученик вводит код → Web SPA отправляет POST → API Gateway проверяет JWT → Code Analyzer строит AST → Error Detector находит ошибку → Hint Generator запрашивает историю ошибок из User Profile → формирует подсказку → возвращает JSON → Web SPA отображает подсветку и всплывающую плашку. Время обработки < 2 сек (KPI T3).

BP-3

Связь компонентов: Процесс реализован совместно BC-1 и BC-3. API Gateway координирует взаимодействие между контекстами.

Поток: Преподаватель отправляет POST → API Gateway проверяет роль через Auth Service → Session Manager генерирует sessionId и invite_code → сохраняет в БД → возвращает invite-ссылку → ученик открывает ссылку → WS Handler проверяет права → добавляет в сессию → рассылает событие join → Awareness Manager синхронизирует курсоры и код между всеми участниками через WSS.

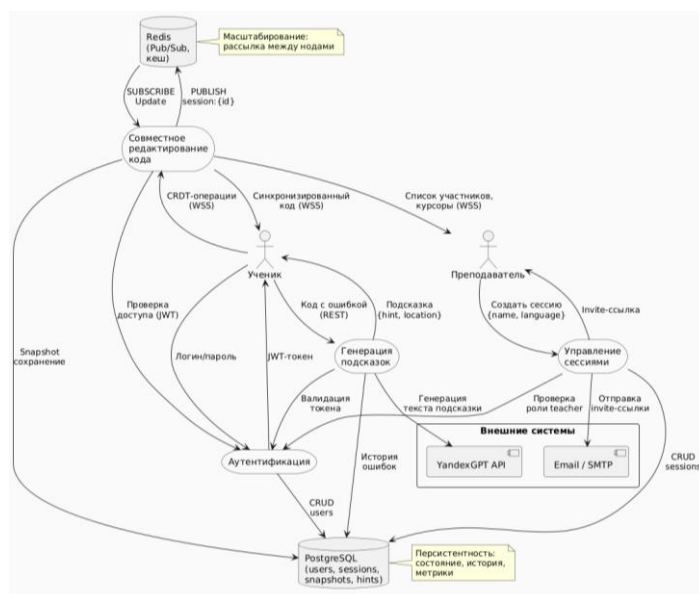


Рисунок 4 – Общая диаграмма (упрощенная) потоков данных системы

1.4 Архитектура системы

1.4.1 Выбранная архитектура

В качестве архитектурного стиля выбран модульный монолит с изолированными сервисами (Modular Monolith / Service-Oriented), с готовностью к переходу к микросервисам.

Требование ТЗ	Решение	Почему не альтернатива
НФТ-2: Горизонтальное масштабирование	Realtime Sync Service в отдельном контейнере + Redis Pub/Sub	В монолите нельзя масштабировать только WebSocket-часть
НФТ-6: Модульность	Три deployable-единицы с чёткими API	Чистый монолит размывает границы доменов
НФТ-7: DDD с BC	Каждый BC — отдельный сервис	Полные микросервисы избыточны для прототипа
ФТ-6: Фоновый AST-анализ	Hint Service (Python) отделён от Realtime Service (Node.js)	Разные рантаймы требуют разных контейнеров
Прототип, малая команда	Общая БД, Docker Compose	Микросервисы с Kafka усложняют поддержку

3 сервиса (Realtime Sync, Hint Service, API Gateway) + SPA + PostgreSQL + Redis. Можно масштабировать Realtime независимо при росте нагрузки, сохраняя простую работу с БД и отладкой.

1.4.2 Архитектурные диаграммы (C4) + ER-диаграмма БД

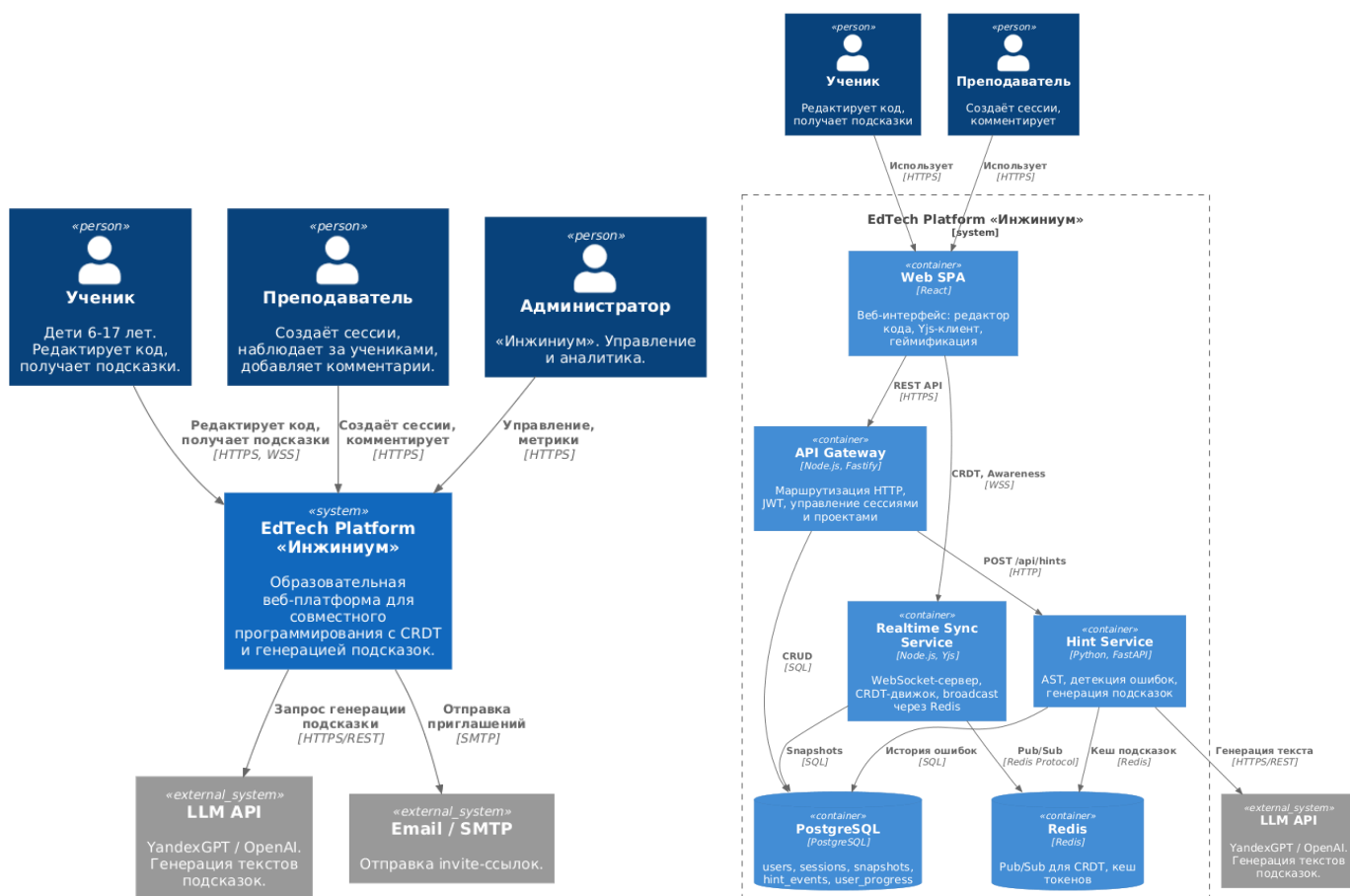


Рисунок 5 – Уровень контекстов C4 (слева) и Уровень контейнеров C4 (справа)

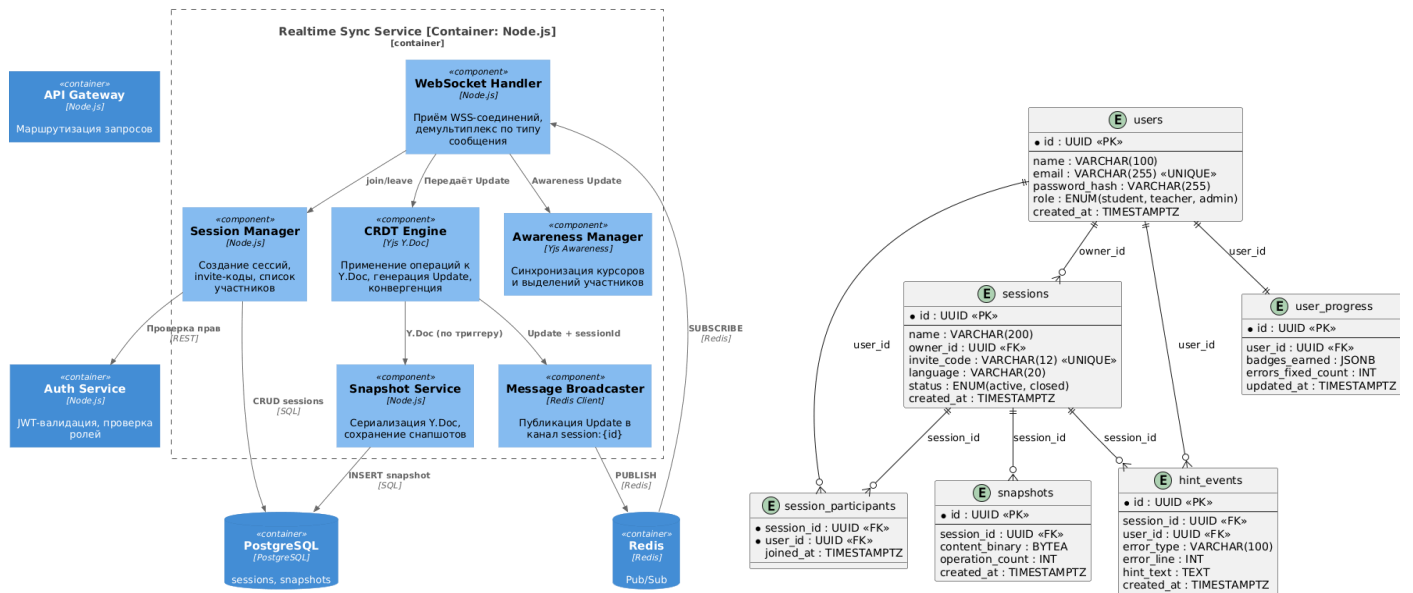


Рисунок 6 – Уровень компонентов C4 (слева) и ER-диаграмма разрабатываемой БД (справа)

1.5 Архитектура инфраструктуры

Система развёртывается в трёх средах: dev (Docker Compose на локальной машине разработчика), test (Kubernetes namespace test), prod (Kubernetes кластер в Yandex Cloud с горизонтальным масштабированием). Все артефакты упаковываются в Docker-образы и хранятся в container registry.

Среда	Платформа	Компоненты	Назначение	Масштабирование
dev	Docker Compose (localhost)	Все сервисы + PostgreSQL + Redis в контейнерах	Локальная разработка, hot reload	Нет
test	Kubernetes	Те же образы, что в prod; отдельная БД test_db	CI-тесты, интеграционные тесты	1 реплика каждого сервиса
prod	Kubernetes (Yandex Cloud) + PostgreSQL + Redis	Стабильные образы	Production	HPA: Realtime 2–10 реплик

1.5.1 Диаграмма развёртывания (Deployment Diagram)

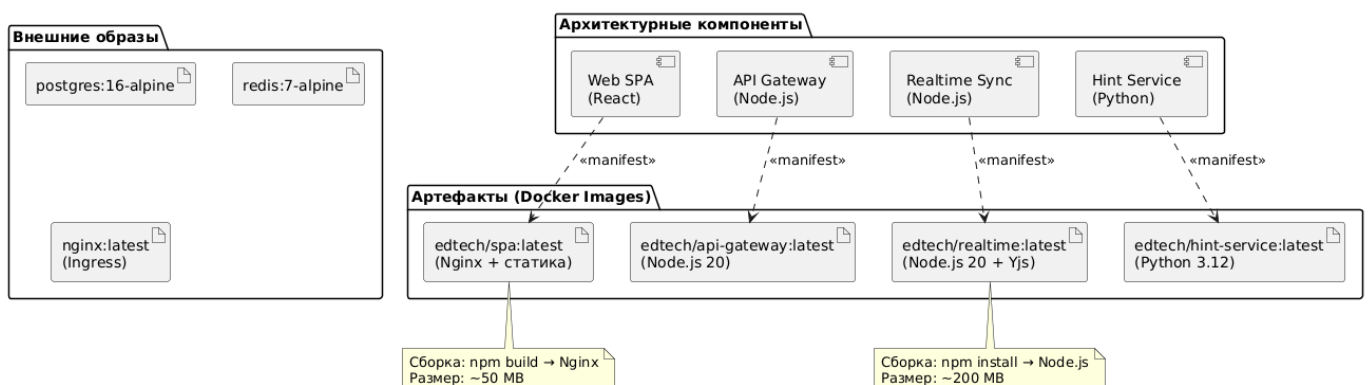


Рисунок 7 – Манифестация компонентов артефактами

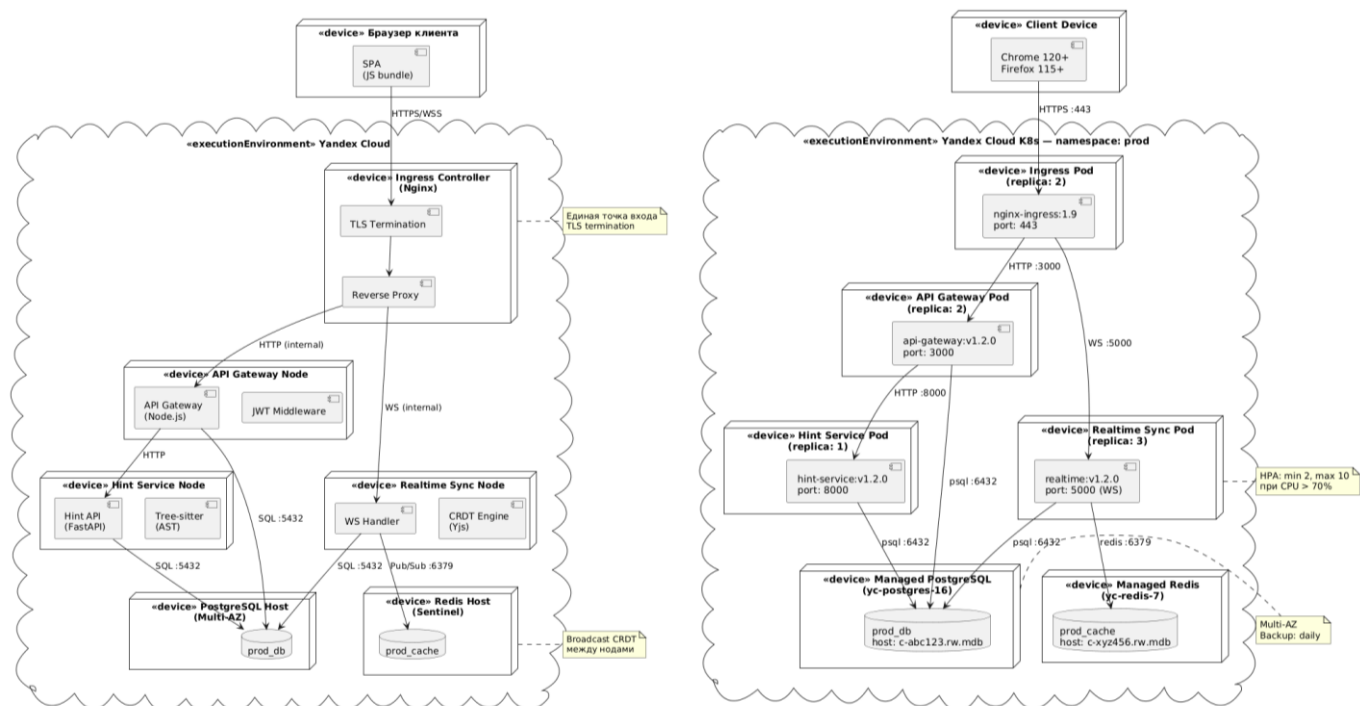


Рисунок 8 – Общая спецификация (уровень типов, слева) и Конкретная спецификация (уровень экземпляров, справа)

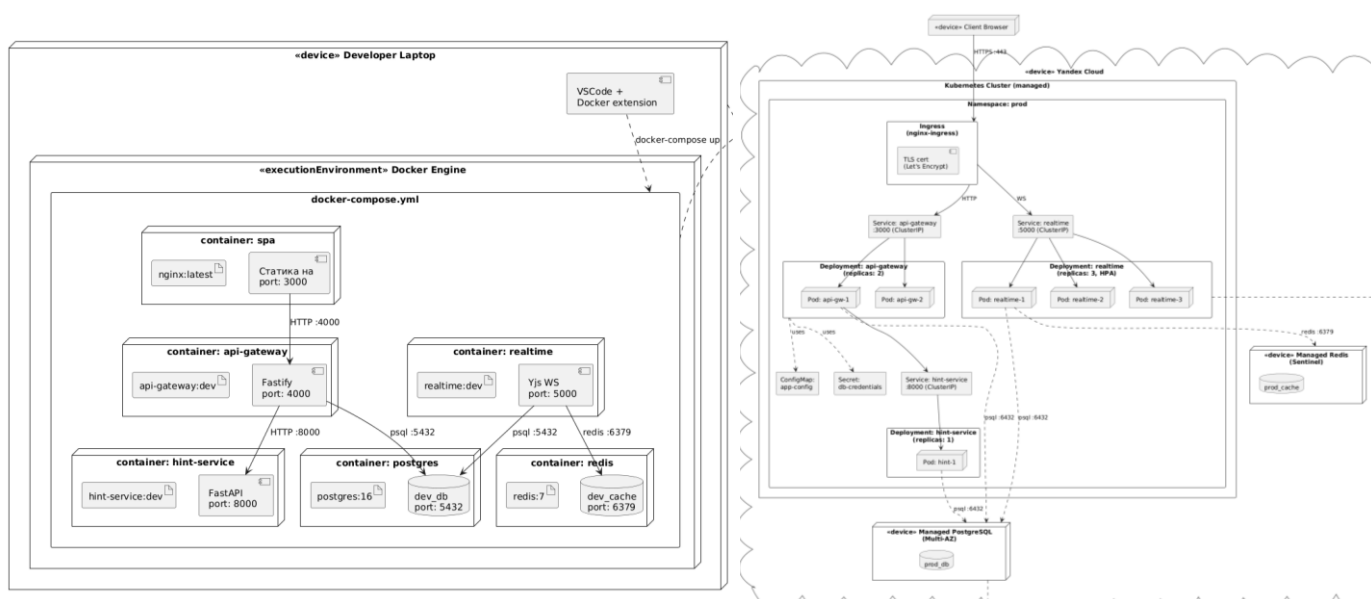


Рисунок 9 – Развертывание в Docker (dev/test, слева) и Развертывание в Kubernetes (prod, справа)

Физическое размещение компонентов (Yandex Cloud, prod):

- Ingress Controller. балансировщик Yandex Cloud
- K8s Worker Nodes. виртуальные машины
- PostgreSQL. Yandex Managed PostgreSQL
- Redis. Yandex Managed Redis
- Container Registry. Yandex Container Registry (хранение Docker-образов)
- Клиентские устройства. браузеры Chrome, Yandex на Windows/macOS/Linux
- Все компоненты находятся в одном VPC для минимизации задержки между сервисами.

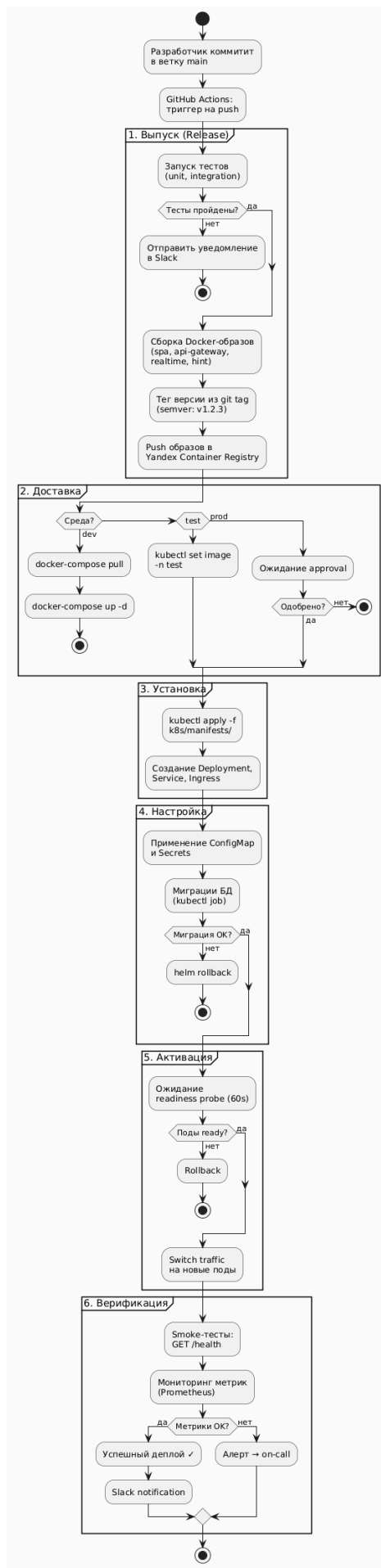


Рисунок 10 – Flowchart разворачивания (CI/CD)

1.6 Технологический стек

Таблица 5 – Выбранный технологический стек с обоснованием

Уровень	Технология	Версия	Обоснование
Frontend	React	18	Компонентная модель, экосистема, SPA
	CodeMirror 6	6.x	Встраиваемый редактор кода в браузере, подсветка Python/JS/Lua, интеграция с Yjs (ФТ-1)
	Yjs	13.x	CRDT (YATA), WebSocket provider, офлайн-буферизация (ФТ-3, НФТ-3)
	TailwindCSS	3.x	Быстрая стилизация, детский дизайн
Realtime Sync	Node.js	20 LTS	Event loop для WS-сервера (НФТ-1)
	y-websocket	1.5.x	WebSocket-провайдер для Yjs, broadcast (ТЗ, разд. 4)
	uWebSockets.js	20.x	Производительный WS-сервер (<100 мс НФТ-1)
API Gateway / LMS	Node.js + Fastify	4.x	HTTP API, маршрутизация, JWT (НФТ-5)
	JWT (jsonwebtoken)	9.x	Stateless-аутентификация (ФТ-12)
Hint Service	Python	3.12	Анализ кода, AST, ML (ФТ-6)
	FastAPI	0.110+	Async REST, OpenAPI (ФТ-15)
	Tree-sitter	0.22+	Инкрементальный парсер Python/JS/Lua (ТЗ, разд. 4)
	YandexGPT / OpenAI API	—	LLM для адаптации подсказок (ФТ-8)
Базы данных	PostgreSQL	16	ACID, JSONB, снапшоты, метрики
	Redis	7	Pub/Sub для CRDT, кеш (НФТ-2)
Инфраструктура	Docker	25.x	Контейнеризация всех сервисов
	Kubernetes	1.31	Оркестрация prod (НФТ-2)
	Nginx Ingress	1.9	TLS termination, reverse proxy
	Yandex Cloud	—	Managed K8s, PostgreSQL, Redis, S3, YandexGPT
CI/CD	GitHub Actions	—	Автосборка, тесты, деплой
	SonarQube	10.x	Статический анализ качества кода

2 Тестирование системы

2.1 Введение

Целью тестирования является верификация соответствия прототипа образовательной платформы функциональным и нефункциональным требованиям перед релизом. А точнее: подтверждение бесконфликтной CRDT-синхронизации, качества и педагогической адекватности подсказок, соблюдения технических метрик и безопасности (НФТ-5).

2.1.1 Область и границы тестирования

Область тестирования: все три Bounded Context (BC-1 Realtime Sync, BC-2 Hint Service, BC-3 API Gateway/LMS), веб-интерфейс SPA, REST API, WebSocket-протокол, интеграция с PostgreSQL и Redis. Границы тестирования: в область не входит внешний LLM API (YandexGPT/OpenAI), тестируется через mock; браузеры, отличные от Chrome и Firefox последних версий; мобильные устройства и touch-интерфейс (вне scope MVP); интеграция с внешней ЛМС «Инжиниум» (перспективный функционал).

2.1.2 Связь тестирования с целями проекта, указанными в ТЗ

Цель / задача из ТЗ	Что покрывает тестирование
Задача 1: Бесконфликтная синхронизация (ФТ-3, НФТ-3)	Интеграционные и нагрузочные тесты CRDT-движка, конвергенция, восстановление после разрыва связи
Задача 2: Генерация понятных подсказок (ФТ-6,7,8)	Unit-тесты Hint Generator, экспертная оценка педагогом (раздел 3.3 ТЗ)
Задача 3: Веб-интерфейс для детской аудитории (ФТ-1, НФТ-4)	E2E-сценарии, юзабилити-тестирование ($SUS \geq 70$ по разделу 3.3 ТЗ)
Задача 4: Низкая задержка (НФТ-1, KPI)	Нагрузочные тесты: задержка < 100 мс при 10 пользователях, время генерации подсказки < 2 с
Ожидаемый результат 2: функционирующий прототип (раздел 1.5 ТЗ)	Системное и приёмочное тестирование сценариев по Use Cases раздела 2 ТЗ
Критерии оценки (раздел 3.3 ТЗ)	Acceptance criteria AC-01–AC-10, проверка функциональной полноты, технических метрик и пользовательских оценок

2.2 Объекты тестирования (Test Items)

TI-ID	Компонент / Модуль	Bounded Context	Функциональные области тестирования	Уровень тестирования
TI-01	CRDT Engine (Yjs Y.Doc)	BC-1 Realtime Sync	Конвергенция при конкурентных вставках/удалениях; идемпотентность операций; корректность YATA-алгоритма	Unit
TI-02	WebSocket Handler	BC-1 Realtime Sync	Установка/разрыв соединений; демультимплексирование сообщений; broadcast всем участникам сессии; задержка < 100 мс	Integration, Load
TI-03	Session Manager	BC-1 / BC-3	Создание сессии; генерация invite-кода; управление участниками (join/leave); персистентность метаданных	Unit, Integration
TI-04	Code Analyzer (Tree-sitter AST)	BC-2 Hint Service	Построение корректного AST для Python и JavaScript; обработка синтаксически некорректного кода без сбоя	Unit
TI-05	Error Detector	BC-2 Hint Service	Обнаружение типовых ошибок начинающих: незакрытые скобки, необъявленные переменные, синтаксические нарушения	Unit
TI-06	Hint Generator	BC-2 Hint Service	Корректность структуры ответа {hint, location, visualCue}; соблюдение детско-адаптированного языка; время ответа < 2 с	Unit, Load
TI-07	Auth Service (JWT)	BC-3 LMS	Регистрация и вход; выдача и валидация JWT; истечение токена; шифрование паролей (bcrypt)	Unit, Security
TI-08	REST API Gateway	BC-3 LMS	Маршрутизация запросов; проверка JWT на всех эндпоинтах; корректные HTTP-статусы; OpenAPI-соответствие	Integration, Security
TI-09	Web SPA (React + CodeMirror)	Клиент	Отображение редактора; подсветка синтаксиса; визуальная привязка подсказок; геймификация (значки, прогресс)	Component, E2E
TI-10	Snapshot Service	BC-1 Realtime Sync	Создание снимков по триггеру; корректность сериализации Y.Doc; API отката к версии	Integration
TI-11	Система (E2E-сценарии)	Все BC	Полный путь ученика (создание сессии → ввод кода → получение подсказки → исправление → геймификация); путь преподавателя	System, Usability

2.3 Тестируемые функции

2.3.1 Основные функции системы, которые будут тестироваться

Функциональная область	Требования ТЗ	Описание	Ожидаемая валидация
Веб-редактор кода	ФТ-1	Текстовый редактор с подсветкой синтаксиса Python и JavaScript	Корректная подсветка; открытие редактора без ошибок
Управление сессиями	ФТ-2, ФТ-13	Создание сессии, invite-ссылка, список «Мои проекты»	Генерация корректной ссылки; сохранение сессии в БД
CRDT-синхронизация	ФТ-3, НФТ-1, НФТ-3	Синхронизация изменений в реальном времени, конвергенция, офлайн-устойчивость	p95-задержка < 100 мс; нулевая потеря данных при разрыве
Присутствие участников	ФТ-4	Список онлайн-пользователей, курсоры и выделения	Курсор участника виден в течение 500 мс
История изменений	ФТ-5	Снапшоты кода, откат к предыдущей версии	Корректный rollback к выбранному снапшоту
AST-анализ кода	ФТ-6	Фоновый статический анализ с построением AST	AST строится без краша для некорректного кода
Обнаружение ошибок	ФТ-7	Выявление типовых ошибок начинающих (скобки, переменные)	Корректный тип и координаты ошибки
Генерация подсказок	ФТ-8, ФТ-9	Детско-адаптированная подсказка, визуальная привязка к строке	Ответ < 2 с; отсутствие технической терминологии
Ручной запрос подсказки	ФТ-10	Кнопка «Помощь» для явного запроса	Подсказка появляется по нажатию
Комментарий преподавателя	ФТ-11	Добавление текстовой аннотации к строке кода	Аннотация видна всем участникам
Аутентификация	ФТ-12, НФТ-5	Регистрация, вход, JWT, шифрование паролей	Корректный JWT; 401/403 при нарушении прав
Геймификация	ФТ-14	Значки за исправление ошибок, прогресс-бары	Значок появляется после факта исправления
Нагрузочная производительность	НФТ-1	10 одновременных пользователей в сессии	p95-задержка WS < 100 мс
Безопасность передачи данных	НФТ-5	HTTPS/WSS на всех эндпоинтах	0 незащищённых соединений; HSTS-заголовок

2.3.1 Исключенные из тестирования на этапе MVP функции

Функция	Требование ТЗ	Причина исключения
RESTful API для внешних систем	ФТ-15	Покрывается OpenAPI-документацией; интеграция с внешними ЛМС запланирована в следующем релизе
Язык Lua в редакторе	ФТ-1	Не входит в MVP-набор языков «Инжиниум»; добавляется в следующей итерации
Горизонтальное масштабирование под нагрузкой 100+ пользователей	НФТ-2	Ограничения тестовой среды (1 нода); проверяется архитектурно через Redis Pub/Sub, полноценный нагрузочный тест — следующая фаза
Проникающее тестирование (Penetration Testing)	НФТ-5	Выходит за рамки MVP; базовая безопасность проверяется OWASP ZAP passive scan и SAST (SonarQube)
Мобильные браузеры и touch-интерфейс	НФТ-4	Целевая платформа MVP — десктопный браузер; мобильная адаптация запланирована в v2.0
Внешний LLM API (реальные вызовы)	ФТ-8	Нестабильность внешнего сервиса; тестируется через mock с фиксированными ответами
Локализация и доступность (ally)	НФТ-4	Не указано явно в ТЗ; оценивается только через SUS-тест с детьми

2.4 Стратегия тестирования

2.4.1 Виды тестирования

Вид тестирования	Объекты	Инструменты
------------------	---------	-------------

Модульное (Unit)	TI-01 CRDT Engine, TI-04 Code Analyzer, TI-05 Error Detector, TI-06 Hint Generator, TI-07 Auth Service	Vitest (Node.js), Pytest (Python)
Интеграционное	TI-02 WS Handler + Redis, TI-03 Session Manager + БД, TI-08 API Gateway + Hint Service, TI-10 Snapshot Service	Supertest (HTTP), ws-client (WebSocket), Testcontainers (PostgreSQL, Redis)
Системное (E2E)	TI-11 — полные пользовательские сценарии (ученик, преподаватель)	Playwright (браузерная автоматизация)
Нагрузочное	TI-02 WS (задержка < 100 мс @ 10 users), TI-06 Hint API (< 2 с)	k6, Prometheus, Grafana
Безопасности	TI-07 Auth, TI-08 API (HTTPS, JWT, SQL Injection)	OWASP ZAP (passive scan), SonarQube (SAST)
Юзабилити	TI-09 SPA, TI-11 E2E — детская аудитория	SUS-опросник, модерируемая сессия
Регрессионное	Unit + Integration suite целиком	GitHub Actions (запуск при каждом PR)

2.4.2 Ручное и автоматизированное тестирование

Тип	Автоматизация	Инструмент	Запуск
Unit	100% авто	Vitest, Pytest	При каждом коммите (CI)
Integration	100% авто	Supertest, Testcontainers	При каждом PR (CI)
E2E / Системное	70% авто + 30% ручное	Playwright	По расписанию + перед релизом
Нагрузочное	100% авто	k6	Перед релизом (stage-среда)
Безопасности (SAST)	100% авто	SonarQube, OWASP ZAP	При каждом PR (CI)
Юзабилити	100% ручное	SUS-анкета, запись экрана	Однократно на прототипе
Приёмочное (с заказчиком)	100% ручное	Чек-лист по ФТ-1...ФТ-15	Перед финальным релизом

Регрессионный прогон (Unit + Integration) запускается автоматически через GitHub Actions при каждом Pull Request в ветку main. Результаты доступны в отчёте GitHub Actions; критические падения блокируют merge.

2.4.3 Уровни и методы тестирования

Метод	Применение	Объекты
White-box	Unit-тесты: проверка внутренней логики CRDT-алгоритма, AST-обхода, JWT-генерации; покрытие ветвей кода	TI-01, TI-04, TI-05, TI-07
Black-box	Интеграционные и E2E-тесты: валидация поведения API и UI по входным/выходным данным без знания реализации	TI-02, TI-08, TI-09, TI-11
Сценарное (Scenario-based)	E2E: полные пользовательские сценарии по Use Cases из раздела 2 ТЗ (ученик вводит код → получает подсказку → исправляет; преподаватель создаёт сессию → приглашает)	TI-11
Граничных значений (Boundary)	Нагрузочное: ровно 10 пользователей (граница НФТ-1); ровно 2 секунды (граница КРП подсказки)	TI-02, TI-06
Негативное тестирование	Security: невалидный JWT, истёкший токен, SQL Injection, доступ к чужой сессии	TI-07, TI-08
Исследовательское (Exploratory)	Юзабилити: дети выполняют задание без инструкций (think-aloud); выявление неочевидных проблем UX	TI-09, TI-11

2.5 Критерии успешности / неуспешности (Acceptance Criteria)

2.5.1 Критерии успешного прохождения

АС-ID	Критерий	Метрика / Условие	Источник в ТЗ	Метод проверки
АС-01	Все 15 функциональных требований реализованы и пройдены	15/15 пунктов чек-листа	ТЗ раздел 3.1	Ручное приёмочное тестирование

АС-02	Задержка синхронизации CRDT-при 10 пользователях	p95 < 100 мс	НФТ-1, KPI	Нагрузочный тест k6
АС-03	Время генерации подсказки	p95 < 2000 мс	KPI ТЗ раздел 1.5	Нагрузочный тест k6
АС-04	Конвергенция CRDT при конкурентных операциях	100% из 100 тест-кейсов — документы идентичны	НФТ-3	Unit-тест Vitest
АС-05	Восстановление после разрыва WS (до 5 мин)	0 потерянных операций	НФТ-3	Интеграционный тест
АС-06	SUS-оценка интерфейса (дети 10–14 лет)	Средний балл ≥ 70	НФТ-4	Юзабилити-тестирование
АС-07	Экспертная оценка понятности подсказок педагогом	$\geq 80\%$ подсказок с оценкой $\geq 4/5$	Раздел 3.3 ТЗ	Экспертная оценка «Инжиниум»
АС-08	Общая удовлетворённость пользователей	Средняя оценка $\geq 4.0 / 5.0$	Раздел 3.3 ТЗ	Пилотное занятие + анкета
АС-09	Все соединения защищены	0 HTTP-эндпоинтов; HSTS-заголовок присутствует	НФТ-5	OWASP ZAP scan
АС-10	Отсутствие критических (P0) дефектов в prod	0 открытых P0-багов на момент релиза	—	Defect tracker
АС-11	Покрытие unit-тестами ключевых модулей	Code coverage $\geq 80\%$ для TI-01, TI-04–TI-07	—	Vitest / Pytest coverage report
Метрика / Условие	Источник в ТЗ	Метод проверки		

Тестирование считается успешно завершённым при одновременном выполнении АС-01 ... АС-11 и прохождении приёмочного тестирования с представителем заказчика («Инжиниум»).

2.5.2 Критерии отказа и приостановки тестирования

Условие	Последствие	Действие
p95-задержка > 150 мс при 10 пользователях (> 50% от порога НФТ-1)	Блокирующий дефект P0	Приостановка релиза; архитектурный ревью Realtime Sync Service
Потеря данных при разрыве WS-соединения (АС-05 не выполнен)	Блокирующий дефект P0	Приостановка; исправление CRDT-буферизации
Обнаружена уязвимость OWASP Top-10 (SQL Injection, Auth Bypass)	Блокирующий дефект P0	Немедленная приостановка; исправление до возобновления
SUS < 50 (критически низкая оценка юзабилити)	Дефект P1	Полный ревью UI/UX; повторное тестирование после редизайна
Более 3 открытых P1-дефектов на момент релиза	Дефект P1	Перенос релиза до устранения
Среда тестирования недоступна или нестабильна	Организационная блокировка	Приостановка тестирования до восстановления среды
Тестовые данные повреждены или несовместимы	Организационная блокировка	Приостановка; восстановление из резервного набора данных

2.6 Результаты тестирования (Test Deliverables)

2.6.1 Ключевые тест-кейсы

ТС-ID	Название	Объект (ТИ)	Приоритет	Предусловия	Входные данные	Шаги выполнения	Ожидаемый результат
-------	----------	-------------	-----------	-------------	----------------	-----------------	---------------------

TC-01	CRDT-конвергенция при одновременном вводе двумя клиентами	TI-01 CRDT Engine	P0 — блокирующий	Два Yjs Y.Doc экземпляра инициализированы одним состоянием: строка "hello" (5 символов)	Клиент А: вставка "!" в позицию 5; Клиент В: одновременная вставка "?" в позицию 5	1. Применить Update от А к Y.Doc_A. 2. Применить Update от В к Y.Doc_B. 3. Обменяться операциями: применить Update_A к Y.Doc_B и Update_B к Y.Doc_A. 4. Сравнить итоговые строки Y.Doc_A и Y.Doc_B.	Оба документа содержат идентичную строку; порядок символов детерминирован алгоритмом CRDT (YATA); расхождений нет
TC-02	CRDT-конвергенция при удалении одного и того же символа	TI-01 CRDT Engine	P0 — блокирующий	Два Y.Doc с текстом "hello"	Клиент А: удаляет символ "l" (позиция 2); Клиент В: удаляет тот же символ "l" (позиция 2)	1. А генерирует Delete-операцию. 2. В генерирует Delete-операцию на ту же позицию. 3. Оба клиента применяют оба Update. 4. Сравнить содержимое документов.	Символ удалён ровно один раз; итоговый текст "helo" одинаков у обоих; дублирования удаления нет (idempotency)
TC-03	Восстановление сессии после разрыва WS-соединения	TI-02 WS Handler, TI-10 Snapshot Svc	P0 — блокирующий	Два клиента подключены к активной сессии; документ содержит 20 символов	Клиент А вводит 5 символов до разрыва; Клиент А вводит ещё 3 символа во время разрыва (буферизация)	1. Зафиксировать исходное состояние документа у обоих. 2. Принудительно разорвать WS-соединение клиента А (tc netem / kill socket). 3. Клиент А вводит 3 символа. 4. Клиент В вводит 2 символа независимо. 5. Восстановить соединение клиента А (< 5 мин). 6. Дождаться синхронизации. 7. Сравнить документы А и В.	Документы А и В идентичны; все 5 символов (3 от А + 2 от В) присутствуют; данные не потеряны; UI отображал состояние «Переподключение...» во время разрыва
TC-04	Измерение p95-задержки синхронизации при 10 пользователях	TI-02 WS Handler	P0 — блокирующий	10 k6 Virtual Users подключены к одной сессии через WSS; Prometheus + Grafana настроены на сбор метрик	60 сек прогрев + 180 сек стабильная нагрузка; каждый VU отправляет 1 CRDT-операцию/сек (итого ~10 ops/s)	1. Запустить k6-скрипт с 10 VU. 2. Каждый VU замеряет время отправки Update до получения broadcast от сервера (ws_roundtrip_ms). 3. Собрать 1000+ измерений. 4. Вычислить p50, p95, p99 латентности. 5. Зафиксировать CPU/RAM Realtime Service.	p95 < 100 мс; p99 < 200 мс; 0 ошибок подключения; CPU сервиса < 70%
TC-05	Обнаружение синтаксической ошибки (незакрытая скобка, Python)	TI-04 Code Analyzer, TI-05 Error Detector	P1 — критический	Hint Service запущен; Tree-sitter Python grammar загружен	print("Hello" (отсутствует закрывающая скобка))	1. Вызвать analyze_code(code) напрямую. 2. Получить AST через Tree-sitter. 3. Запустить Error Detector. 4. Проверить структуру результата.	Возвращён объект {"error_type": "syntax/unmatched_paren", "location": {"line": 1, "col": 14}, "detected": true}; error_type входит в утверждённый справочник типов ошибок
TC-06	Обнаружение семантической ошибки (использование необъявленной переменной)	TI-04 Code Analyzer, TI-05 Error Detector	P1 — критический	Hint Service запущен	print(x) при отсутствии предшествующего x = ...	1. Вызвать analyze_code(code). 2. Получить AST. 3. Запустить Error Detector с symbol-table анализом. 4. Проверить результат.	Возвращён объект с "error_type": "semantic/undefined_variable", "symbol": "x", "location.line": 1; детекция корректна
TC-07	Генерация подсказки: детско-адаптированный язык	TI-06 Hint Generator	P1 — критический	Hint Generator инициализирован; LLM-клиент замочкован (mock возвращает фиксированный ответ)	{"error_type": "syntax/unmatched_paren", "user_age": 10, "language": "python"}	1. Вызвать generate_hint(error_ctx). 2. Получить JSON-ответ. 3. Проверить поля message, location, visualCue. 4. Прогнать message через список запрещённых терминов (AST, токен, парсинг, компилятор).	Ответ содержит все поля; visualCue == "highlight"; message не содержит ни одного из технических терминов из запрещённого списка; длина message ≤ 150 символов
TC-08	Время отклика Hint Service под нагрузкой	TI-06 Hint Generator	P1 — критический	Hint Service запущен в Docker; LLM API доступен (или stub с фиксированной задержкой 300 мс)	20 параллельных HTTP POST /api/hints с валидным кодом Python	1. k6 отправляет 20 VU × 30 запросов за 60 сек. 2. Замерить http_req_duration для каждого запроса. 3. Вычислить p95.	p95 < 2000 мс; 0 HTTP 5xx ответов; тело ответа содержит валидный JSON с полем hint

ТС-09	Аутентификация: регистрация и получение JWT	TI-07 Auth Service	P1 — критический	API Gateway запущен; тестовая БД PostgreSQL чистая	{ "email": "test@test.ru", "password": "Test1234!", "name": "Иван", "role": "student" }	1. POST /api/auth/register с входными данными. 2. Проверить HTTP 201, тело ответа. 3. POST /api/auth/login с теми же email/password. 4. Проверить HTTP 200 и наличие access_token в теле. 5. Декодировать JWT, проверить поля sub, role, exp.	Шаг 2: HTTP 201, { "id": "uuid", "email": "test@test.ru" }; Шаг 4: HTTP 200, access_token валиден; Шаг 5: role == "student", exp — текущее время + 24 ч
ТС-10	Несанкционированный доступ к чужой сессии	TI-07 Auth Service	P0 — блокирующий	Созданы пользователи User_A и User_B; User_A создал закрытую сессию session_id_A; у User_B есть валидный JWT	Запросы: (а) без токена, (б) с истёкшим JWT, (в) с JWT User_B к сессии User_A	1. GET /api/sessions/session_id_A без заголовка Authorization. 2. Повторить с токеном, у которого exp < now. 3. Повторить с валидным токеном User_B (чужая сессия).	Шаг 1: HTTP 401 { "error": "Unauthorized" }; Шаг 2: HTTP 401 { "error": "Token expired" }; Шаг 3: HTTP 403 { "error": "Forbidden" }
ТС-11	E2E: полный сценарий ученика (создание, ввод, подсказка)	TI-11 (все компоненты)	P1 — критический	Приложение развёрнуто в тестовой среде; создан тестовый аккаунт ученика	Браузер Chrome, учётные данные тестового ученика	1. Открыть главную страницу, войти в аккаунт. 2. Нажать «Создать сессию», выбрать Python. 3. В редакторе ввести print("Привет" (без)). 4. Подождать 2 сек (дебаунс). 5. Проверить появление визуальной подсказки. 6. Нажать кнопку «Помощь». 7. Проверить текст подсказки. 8. Исправить код: добавить). 9. Проверить появление анимации/значка геймификации.	Шаг 1: редирект на дашборд; Шаг 2: открылась сессия с CodeMirror 6; Шаг 5: строка 1 подсвечена, всплывающая плашка появилась; Шаг 7: текст понятный, без жаргона; Шаг 9: анимация «Отлично!» или значок
ТС-12	E2E: сценарий преподавателя (приглашение + наблюдение)	TI-03 Session Mgr, TI-11	P1 — критический	Аккаунты преподавателя и двух учеников созданы	Два браузера (преподаватель + ученик), invite-ссылка	1. Преподаватель создаёт сессию, копирует invite-ссылку. 2. Ученик открывает ссылку, присоединяется. 3. Проверить отображение курсора ученика у преподавателя. 4. Ученик вводит 3 строки кода. 5. Преподаватель добавляет текстовый комментарий к строке 2. 6. Проверить отображение комментария у ученика.	Шаг 3: аватар/курсор ученика виден в реальном времени; Шаг 4: код у преподавателя обновляется без задержки; Шаг 6: аннотация преподавателя отображается у ученика в течение 500 мс
ТС-13	Откат к снапшоту	TI-10 Snapshot Service	P2 — важный	Сессия с историей из 50+ операций; снапшот создан автоматически на 30-й операции	API-запрос на откат к снапшоту №1	1. Зафиксировать текущее состояние документа. 2. GET /api/sessions/{id}/snapshots — получить список. 3. POST /api/sessions/{id}/rollback с { "snapshot_id": snap_1 }. 4. Получить актуальный документ. 5. Сравнить с содержимым снапшота.	HTTP 200; документ совпадает с содержимым снапшота на момент 30-й операции; операции 31–50 откатены
ТС-14	Проверка HTTPS/WSS: отсутствие незащищённых соединений	TI-07, TI-08 (инфраструктура)	P0 — блокирующий	Приложение развёрнуто в test-среде с TLS-сертификатом	OWASP ZAP passive scan на всех эндпоинтах	1. Запустить ZAP passive spider по базовому URL. 2. Проверить все обнаруженные ресурсы на протокол. 3. Проверить заголовки: Strict-Transport-Security, X-Content-Type-Options, Content-Security-Policy. 4. Попытаться открыть HTTP-версию — убедиться в редиректе на HTTPS.	0 ресурсов по HTTP; HSTS-заголовок присутствует; HTTP-запрос возвращает 301 → HTTPS; ZAP: 0 High-risk алертов

ТС-15	Юзабилити: SUS-тест с детьми	TI-09 SPA, TI-11	P1 — критический	5–7 детей возраста 10–14 лет; каждому выдан чистый аккаунт; модератор зафиксировал задание	Задание: «Открой платформу, создай сессию, напиши print("Привет" без скобки, найди подсказку и исправь ошибку»	1. Ребёнок выполняет задание самостоятельно (think-aloud). 2. Фиксировать время на задание, количество обращений за помощью. 3. После выполнения — заполнить SUS-анкету (10 вопросов по шкале 1–5). 4. Рассчитать SUS-балл по формуле: $(\sum \text{нечётных} - 5 + 25 - \sum \text{чётных}) \times 2.5$.	Средний SUS ≥ 70 ; ни один ребёнок не обратился за помощью более 3 раз; 100% детей нашли кнопку «Помощь» без подсказки модератора
-------	------------------------------	------------------	------------------	--	--	--	--

Таблица 6 – Матрица трассировки тест-кейсов к требованиям

Требование ТЗ	Покрывающие ТС	Тип тестирования
ФТ-1 (редактор с подсветкой)	ТС-11	E2E
ФТ-2 (создание сессии / invite)	ТС-11, ТС-12	E2E
ФТ-3 (CRDT-синхронизация)	ТС-01, ТС-02, ТС-03, ТС-04	Unit, Integration, Load
ФТ-4 (курсоры участников)	ТС-12	E2E
ФТ-5 (история / откат)	ТС-03, ТС-13	Integration
ФТ-6 (AST-анализ в фоне)	ТС-05, ТС-06	Unit
ФТ-7 (обнаружение ошибок)	ТС-05, ТС-06	Unit
ФТ-8 (генерация подсказок)	ТС-07, ТС-08	Unit, Load
ФТ-9 (визуальная привязка подсказки)	ТС-07, ТС-11	Unit, E2E
ФТ-10 (ручной запрос «Помощь»)	ТС-11	E2E
ФТ-11 (комментарий преподавателя)	ТС-12	E2E
ФТ-12 (аутентификация)	ТС-09, ТС-10	Integration, Security
ФТ-14 (геймификация)	ТС-11	E2E
НФТ-1 (< 100 мс @ 10 users)	ТС-04, ТС-12	Load
НФТ-3 (офлайн-устойчивость)	ТС-03	Integration
НФТ-4 (детский UI, SUS ≥ 70)	ТС-07, ТС-11, ТС-15	Unit, E2E, Usability
НФТ-5 (HTTPS/WSS)	ТС-10, ТС-14	Security
KPI: подсказка < 2 сек	ТС-08	Load

2.6.2 Шаблон отчета об ошибках (defect reports)

Поле	Описание
Defect ID	DEF-XXX (сквозная нумерация)
Название	Краткое однострочное описание
Компонент	TI-01 ... TI-11
Severity	P0 Blocker / P1 Critical / P2 Major / P3 Minor
Priority	High / Medium / Low
Шаги воспроизведения	Пронумерованная последовательность
Ожидаемый результат	Поведение по требованию ТЗ
Фактический результат	Наблюдаемое поведение
Среда	Dev / Test / Prod; версия сборки (git tag)
Вложения	Скриншот, HAR-файл, лог сервера
Статус	Open / In Progress / Fixed / Verified / Closed

Критерий закрытия дефекта: повторное выполнение соответствующего ТС дало статус Pass; регрессионный прогон не выявил новых падений.

2.6.3 Итоговый отчёт о качестве системы

Формируется по завершении всех фаз тестирования и содержит следующее:

Раздел отчёта	Содержание
Сводка по Acceptance Criteria	Таблица AC-01 ... AC-11: статус Pass / Fail по каждому критерию
Статистика тест-кейсов	Passed / Failed / Skipped; % выполнения по уровням (Unit, Integration, E2E)
Метрики нагрузочного теста	Графики p50/p95/p99 latency (WS и Hint API); CPU/RAM сервисов

Результаты юзабилити	Средний SUS-балл; детализация по 10 вопросам анкеты
Экспертная оценка подсказок	Доля оценок $\geq 4/5$ от педагога «Инжиниум»; типичные замечания
Реестр дефектов	Все DEF-XXX с severity, статусом и ссылкой на коммит-фикс
Покрывание кода	Code coverage (%) по модулям TI-01, TI-04–TI-07
Заключение Go / No-Go	Обоснованное решение о готовности к релизу на основе AC-01...AC-11

Отчёт утверждается руководителем курсовой и представителем заказчика («Инжиниум») как подтверждение соответствия системы критериям раздела 3.3 ТЗ.

3 Список источников

1. Shapiro, M. Conflict-Free Replicated Data Types [Текст] / M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski // Lecture Notes in Computer Science. — 2011. — P. 386–400. — DOI: [10.1007/978-3-642-24550-3_29](https://doi.org/10.1007/978-3-642-24550-3_29).
2. Delev, T. Static analysis of source code written by novice programmers [Текст] / T. Delev, D. Gjorgjevikj // 2017 IEEE Global Engineering Education Conference (EDUCON). — 2017. — P. 825–830. — DOI: [10.1109/EDUCON.2017.7942942](https://doi.org/10.1109/EDUCON.2017.7942942).
3. Crow, T. Intelligent tutoring systems for programming education [Текст] / T. Crow, A. Luxton-Reilly, B. Wuensche // Proceedings of the 20th Australasian Computing Education Conference (ACE '18). — 2018. — DOI: [10.1145/3160489.3160492](https://doi.org/10.1145/3160489.3160492).
4. Vernon, V. Domain-Driven Design Distilled [Электронный ресурс] / V. Vernon. — URL: <https://cdn.bookekey.app/files/pdf/book/en/domain-driven-design-distilled.pdf> (дата обращения: 03.02.2026).
5. Эванс, Э. Предметно-ориентированное проектирование (DDD): структуризация сложных программных систем [Текст] / Э. Эванс. — Москва: Вильямс, 2011. — 448 с.
6. Инжиниум: собственная экосистема обучения [Электронный ресурс] // Проектная ИТ-школа «Инжиниум». — URL: <https://inginium.ru/> (дата обращения: 03.02.2026).
7. Computer-supported collaborative learning [Электронный ресурс] // Wikipedia. — 2025. — URL: https://en.wikipedia.org/wiki/Computer-supported_collaborative_learning (дата обращения: 03.02.2026).
8. Pair programming [Электронный ресурс] // Wikipedia. — 2020. — URL: https://en.wikipedia.org/wiki/Pair_programming (дата обращения: 03.02.2026).
9. Project-based learning [Электронный ресурс] // Wikipedia. — 2019. — URL: https://en.wikipedia.org/wiki/Project-based_learning (дата обращения: 03.02.2026).
10. Психологи онлайн на Ясно [Электронный ресурс] // yasno.live. — URL: <https://yasno.live/> (дата обращения: 03.02.2026).
11. Шкала Лайкерта [Электронный ресурс] // Википедия. — 2009. — URL: https://ru.wikipedia.org/wiki/Шкала_Лайкерта (дата обращения: 03.02.2026).
12. System usability scale [Электронный ресурс] // Wikipedia. — 2019. — URL: https://en.wikipedia.org/wiki/System_usability_scale (дата обращения: 03.02.2026).
13. A Guide to the Project Management Body of Knowledge (PMBOK® Guide) [Текст] / Project Management Institute. — 6th ed. — 2017.

Для редактирования текста и проверки на наличие лексических и грамматических ошибок использованы следующие языковые модели:

14. ChatGPT (версия от 14 марта 2023) [Большая языковая модель] / OpenAI. — 2023. — URL: <https://chat.openai.com/> (дата обращения: 03.02.2026).
15. DeepSeek Chat [Большая языковая модель] / DeepSeek. — URL: <https://chat.deepseek.com/> (дата обращения: 03.02.2026).